FastAPI + Pydantic + SQLAlchemy — My Learning Notes

> Practice project: a simple **Product** CRUD API built with FastAPI, validated with Pydantic, and stored in PostgreSQL through SQLAlchemy.

Project files

File	Purpose
`main.py`	FastAPI app, all endpoints, DB seeding, dependency injection
`models.py`	Pydantic models (data validation / JSON shape)
`dbmodels.py`	SQLAlchemy models (database schema / tables)
`dbConfig.py`	Database URL, engine, session factory
`requirements.txt`	Project dependencies

Table of Contents

---

1. FastAPI

- Is a **web framework**.

---

2. Uvicorn

Like for React there is Node, for PHP there is XAMPP, and here **Uvicorn** will be used.

Uvicorn is the server that actually runs the FastAPI application.

---

3. My First Endpoint

When a user enters the website you want it to return something. So in Python, for this, we make a function.

So inside `main.py`:

```
def greet():
    return "Welcome to my website"
```

And now we can use our web server Uvicorn to run this, using the `uvicorn main --reload` command.

```
ERROR:    Error loading ASGI app. Import string "main" must be in format "
<module>:<attribute>".
```

The error mentions that the module is there, but where is the attribute? We are using FastAPI but not mentioning it anywhere. So we import FastAPI, make its object, and then run our server with the attribute.

`main.py`:

```
from fastapi import FastAPI

app = FastAPI()

def greet():
    return "Welcome to my website"
```

Command:

```
uvicorn main:app --reload
```

For now it will provide us with `{"detail":"Not Found"}` because we have not specified endpoints.

---

4. REST API

So the web client sends a request to the backend, the backend examines and executes the request with the database, and then sends the response back to the client. Now it can't be just any command — it needs to follow a standardized structure so both sides perfectly understand each other.

REST (Representational State Transfer) provides this predictable rulebook. Instead of inventing custom instructions for every single action, REST treats everything as a **resource** (like a user, a blog post, or a product) and relies on standard HTTP methods to interact with them:

- **GET:** Read or fetch a resource.
- **POST:** Create a new resource.
- **PUT / PATCH:** Update an existing resource.
- **DELETE:** Remove a resource.

Because these rules are universal, any client — whether it's a web browser, a mobile app, or a smart TV — can communicate seamlessly with the backend without needing a custom translation manual.

So now in my `main.py` I can use the GET method to read:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def greet():
    return "Welcome to my website"
```

Now our website will show `"Welcome to my website"`.

---

5. Pydantic

Pydantic is for **data validation**, and if you want to convert data from the server side to JSON format to send it to the client, we will use this.

For example, a user enters a negative value for age or price, or maybe a 2-character name — this is not acceptable, although they will still satisfy the data type. So for such data validation we use Pydantic.

So inside our `models.py`:

Reference code — `models.py` (complete file):

```
from pydantic import BaseModel  # Import base model

class Product(BaseModel):  # Inherit base model
    id: int
    name: str
    description: str
    price: float
    quantity: int

# Now we dont need this constructor after pydantic (Commented out)

    # def __init__(self, id: int, name: str, description: str, price: float,
quantity: int):
```

```
    #     self.id = id
    #     self.name = name
    #     self.description = description
    #     self.price = price
    #     self.quantity = quantity
```

> **Note:** Pydantic builds the constructor for us from the type hints, which is why the manual `__init__` is no longer needed.

The dummy product list (`main.py`)

This is the in-memory list I started with, before the database existed:

```
products = [
    Product(id=1, name="Phone", description="A smartphone", price=699.99,
quantity=50),
    Product(id=2, name="Laptop", description="A powerful laptop", price=999.99,
quantity=30),
    Product(id=3, name="Pen", description="A blue ink pen", price=1.99,
quantity=100),
    Product(id=4, name="Table", description="A wooden table", price=199.99,
quantity=20),
]
```

---

6. Swagger UI

In FastAPI we have **Swagger UI** by default to test out our backend. Just add `/docs` at the end of the URL.

```
http://127.0.0.1:8000/docs
```

---

7. CRUD Operations — In-Memory List Version

This is the first version, before SQLAlchemy. These are now commented out in `main.py`, but kept here for reference.

Read All

```
@app.get("/products")
def getAllProducts():
    # db = Session()  # DB Connection
    # db.query()      # Then, execute queries
    return products
```

Dynamic data fetching — use {}

```
@app.get("/product/{id}")
def getProductById(id: int):
    for product in products:
        if product.id == id:
            return product

    return "Product not found"
```

Data adding using POST

```
@app.post("/product")
def addProduct(product: Product):
    products.append(product)
    return product
```

Update Method

```
@app.put("/product")
def updateProduct(id: int, product: Product):
    for i in range(len(products)):
        if products[i].id == id:
            products[i] = product
            return "Product added successfully"
    return "No product Found"
```

Delete Method

```
@app.delete("/product")
def deleteProduct(id: int):
    for i in range(len(products)):
        if products[i].id == id:
            del products[i]
            return "Product Deleted"
    return "Product Not found"
```

---

8. SQLAlchemy

SQLAlchemy is an open-source library used to interact with relational databases using Python code instead of raw SQL. It functions as both a **database toolkit** and an **Object-Relational Mapper (ORM)**, allowing developers to map database tables directly to Python classes.

---

9. DB Configuration (`dbConfig.py`)

Creating an object of `SessionLocal` and then we can use it in our app for the database connection.

- Every time you connect to something, that is a **session**.
- To create a session we have `sessionmaker`.

```
from sqlalchemy.orm import sessionmaker
from sqlalchemy import create_engine
```

`sessionmaker` parameters

- **autocommit** — Whenever we do database transactions, normally we have to do commits for it, but by default it will be `autocommit=False` and we can set it to `False` or `True`.
- **autoflush** — Autoflush is a configuration setting that forces the Session to automatically write pending in-memory data changes to the database transaction immediately before any new query is executed.
- **bind** — Passing `bind=engine` into `sessionmaker` links your database driver configurations directly to your session factory. This setting ensures that every individual Session instance generated by that factory automatically knows which database to communicate with and where to request connection resources.

When `Session = sessionmaker(bind=engine)` is called, you are establishing a default database target. Whenever a newly generated session needs to execute a query or flush data, it silently goes to that specific engine, checks out a physical connection, runs the SQL, and returns the connection to the pool when done.

Reference code — `dbConfig.py` (complete file):

```
# Creating object of SessionLocal and then we can use it in our app for database
connection
# Everytime you connect to something that is a session
# to create a session we have sessionmaker
from sqlalchemy.orm import sessionmaker
from sqlalchemy import create_engine

db_url = "postgresql://postgres:12345678@localhost:5432/FASTAPI_Practice"

engine = create_engine(db_url)

Session = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

Breaking down the DB URL

```
postgresql://postgres:12345678@localhost:5432/FASTAPI_Practice
   dialect     user   password   host   port     database name
```

How to use it

Go to where you want to use your DB (`main.py` in this case):

1. Import the database session from `dbConfig` using `from dbConfig import Session`.
2. Make a DB session `db = Session()` in any function inside endpoints.
3. Execute a query by using `db.query(<model>)`.

We don't need to write actual SQL queries because of SQLAlchemy.

---

10. SQLAlchemy Models (`dbmodels.py`)

Now for mapping the database and the class. The classes in `models.py` are Pydantic, so we have to create another kind of class based on SQLAlchemy for the database. So the database schema will be created based on that particular class.

We can mention column types, name, and some filters or special access like Foreign keys, Primary keys, etc. We can't do that inside Pydantic base model classes.

So we need:

Class type	File	Job
Pydantic (`BaseModel`)	`models.py`	Data validation + JSON conversion
SQLAlchemy (`Base`)	`dbmodels.py`	Database table / schema

My Pydantic models are in `models.py` and SQLAlchemy models in `dbmodels.py` for now.

- We will use **`Base` of SQLAlchemy** instead of **`BaseModel` of Pydantic**.
- So we use `= Column(<define attr>)` to define each field.

Reference code — `dbmodels.py` (complete file):

```
# We will use Base of SQL Alchemy instead of BaseModel of Pydantic
from sqlalchemy import Column, Integer, String, Float
from sqlalchemy.orm import declarative_base

Base = declarative_base()

# So we use = Column(<define attr>) to define

class Product(Base):

    __tablename__ = "product"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    description = Column(String)
```

```
    price = Column(Float)
    quantity = Column(Integer)
```

> `__tablename__` is the actual table name that will be created in PostgreSQL. `primary_key=True` makes `id` unique, `index=True` makes lookups on `id` faster.

---

11. Creating the Tables

Now in `main.py` we can create tables according to our models in `dbmodels.py` and import `engine` from `dbConfig`.

```
import dbmodels
```

Then I will say in `main.py` that "hey, use the metadata in `dbmodels` to create tables and bind to engine":

```
from dbConfig import Session, engine
import dbmodels

app = FastAPI()

dbmodels.Base.metadata.create_all(bind=engine)
```

---

12. Inserting Dummy Data (`db_init`)

Now I want to insert data into my PostgreSQL using SQLAlchemy.

So I want to make a condition that whenever I open my application it should check if the table is empty, populate it with dummy data, and if there is any data then don't do it.

To achieve that we will create a function (`main.py` for now):

```
def db_init():
    db = Session()  # first we need connection object

    for product in products:  # Adding products
        db.add(product)       # add products in it

db_init()
```

Problem 1 — Pydantic object vs SQLAlchemy object

Now here we will have a problem. The DB is working with the database model class of Product which is connected to SQLAlchemy, but in the code the product we are using is not the SQLAlchemy product — it is

actually the Pydantic model. So this will not work. We need to somehow convert this to a SQLAlchemy product. We cannot pass an object of `Product` (Pydantic model) but a product of SQLAlchemy. How are we going to do it?

Let's rewrite:

We need to convert the product in the for loop to a SQLAlchemy product. So we can pass it to `dbmodels.Product(...)`. Now the model in `dbmodels.py`, the `Product(Base)`, accepts key-value pairs for converting it and will create an object for it. Now how to pass key-value pairs?

We can use `.model_dump()` on our Pydantic model to convert it into a dictionary and use Python's syntax `**` for unpacking it to get raw key-value pairs. So it becomes:

```
db.add(dbmodels.Product(**product.model_dump()))
```

Problem 2 — Nothing is saved

In our `dbConfig.py`, `autocommit = False`, so that is why it won't execute — so we need `db.commit()`.

```
def db_init():
    db = Session()  # first we need connection object

    for product in products:
        db.add(dbmodels.Product(**product.model_dump()))
    db.commit()

db_init()
```

Now added data.

> **Note:** We have to make sure that we are adding database entries of SQLAlchemy (`dbmodels.py` in this case), not Pydantic models (`models.py` in this case). We have to convert the Pydantic object using `model_dump()`, which gives a dictionary, and use `**` to unpack it for raw key-value pairs and feed it to the database.

Problem 3 — Duplicate insert on every reload

Now every time we reload the server it will try to again put the same values into the database, but it will give an error because there is a primary key. So how do we fix that?

I don't want it to call this every time I load this. It should do its job only if the table is empty. So basically we can check this before we do that. Added `count` and an `if` condition to track.

To **get** something we use `db.query()`, and for **adding** we use `db.add()`. So for getting the count of rows to track, we will use `db.query(dbmodels.Product).count()` and store it in a `count` variable.

Reference code — `main.py`:

```
def db_init():
    db = Session()

    count = db.query(dbmodels.Product).count
    if count == 0:
        for product in products:
            db.add(dbmodels.Product(**product.model_dump()))
        db.commit()

db_init()
```

> ⚠️ **Correction to remember:** in the code above `.count` is missing its parentheses. `db.query(...).count` returns the *method itself* (always truthy, never equal to `0`), so the seeding block never runs. The correct line is:
>
> ```
> count = db.query(dbmodels.Product).count()
> ```

> **Note:** In a normal project we don't need to do this, because data might already be there or we might add data manually.

---

13. Dependency Injection

Now we have another problem: every time we need to use the database we have to repeatedly write:

```
db = Session()
```

We are repeatedly creating a session and not closing it. Not a good idea.

We should have one place where, when we need it, we will use it and then the connection will be closed.

> **Note:** The `yield` keyword in Python is used to turn a standard function into a **generator function**. Instead of computing all values at once and returning them as a finished list, `yield` produces data lazily — one item at a time — on demand.

So we can make a function for getting the database:

```
def get_db():
    db = Session()
    yield db
    db.close()
```

Imagine if something goes wrong inside `yield db`. I still want to close the connection, not just when `yield db` works fine. So for that we will add `try` and `finally` blocks (error handling).

Reference code — `main.py`:

```
def get_db():
    db = Session()
    try:
        yield db
    finally:
        db.close()
```

Dependency injection (DI) in Python is used to pass external resources — such as database connections, configuration settings, or API clients — directly into a class or function rather than having the code create them internally.

How to inject it

Now how are we going to make use of `get_db()` using dependency injection?

For example, I will use it in `getAllProducts()`. `getAllProducts()` needs to **ask for** the dependency — it will not be given by default.

First import `Session` of `sqlalchemy.orm` (not the local `Session` we created) and also import `Depends` from FastAPI.

> **Note:** If `Session` of `sqlalchemy.orm` conflicts, we can use the `as` keyword and rename it to anything, then use that — for example `from sqlalchemy.orm import Session as DBSession`.

```
from fastapi import Depends
from sqlalchemy.orm import Session as DBSession
```

I want `db` of type Session and this depends on `get_db`:

```
def getAllProducts(db: DBSession = Depends(get_db)):
```

This is how the database is injected into the function, and now we can use our database in this function.

How to use it (example: getting all products)

```
@app.get("/products")
def getAllProducts(db: DBSession = Depends(get_db)):
    db_products = db.query(dbmodels.Product).all()

    return db_products
```

So in this line `db_products = db.query(dbmodels.Product).all()` we created an object `db_products` and used `query`, and specified which model I need in the bracket arguments `db.query(dbmodels.Product)` which is the SQLAlchemy model. And I need all of them, so `.all()`.

---

14. CRUD Operations — Database Version

(Read already done above) — all of these use **database dependency injection and SQLAlchemy**.

Read (all)

```
@app.get("/products")
def getAllProducts(db: DBSession = Depends(get_db)):
    db_products = db.query(dbmodels.Product).all()

    return db_products
```

Specific Read

```
@app.get("/product/{id}")
def getProductById(id: int, db: DBSession = Depends(get_db)):
    db_product = db.query(dbmodels.Product).filter(dbmodels.Product.id ==
id).first()
    if db_product:
        return db_product

    return "Product not found"
```

- `.filter(dbmodels.Product.id == id)` is the `WHERE` clause.
- `.first()` gives back a single row instead of a list.

Create

```
@app.post("/product")
def addProduct(product: Product, db: DBSession = Depends(get_db)):
    db.add(dbmodels.Product(**product.model_dump()))
    db.commit()
    return product
```

Same conversion trick as in `db_init()` — Pydantic in, `model_dump()` → dictionary, `**` unpack → SQLAlchemy object.

Update

```
@app.put("/product")
def updateProduct(id: int, product: Product, db: DBSession = Depends(get_db)):
    db_product = db.query(dbmodels.Product).filter(dbmodels.Product.id ==
id).first()
    if db_product:
        db_product.name = product.name
        db_product.description = product.description
        db_product.price = product.price
        db_product.quantity = product.quantity
        db.commit()
        return "Product updated Successfully"
    else:
        return "Product Not Found"
```

Here we don't create a new object — we fetch the existing row, change its attributes, and `db.commit()` writes the changes. **Without `db.commit()` the update is lost**, because `autocommit=False`.

Delete

```
@app.delete("/product")
def deleteProduct(id: int, db: DBSession = Depends(get_db)):
    db_product = db.query(dbmodels.Product).filter(dbmodels.Product.id ==
id).first()
    if db_product:
        db.delete(db_product)
        db.commit()
        return "Product Deleted"
    else:
        return "Product Not found"
```

---

15. Full Reference Files

`requirements.txt`

```
fastapi
uvicorn
sqlalchemy
psycopg2
```

- `psycopg2` is the PostgreSQL driver that SQLAlchemy talks through.

Install with:

```
pip install -r requirements.txt
```

models.py

```
from pydantic import BaseModel  # Import base model

class Product(BaseModel):  # Inherit base model
    id: int
    name: str
    description: str
    price: float
    quantity: int

# Now we dont need this constructor

    # def __init__(self, id: int, name: str, description: str, price: float,
quantity: int):
    #     self.id = id
    #     self.name = name
    #     self.description = description
    #     self.price = price
    #     self.quantity = quantity
```

dbmodels.py

```
# We will use Base of SQL Alchemy instead of BaseModel of Pydantic
from sqlalchemy import Column, Integer, String, Float
from sqlalchemy.orm import declarative_base

Base = declarative_base()

# So we use = Column(<define attr>) to define

class Product(Base):

    __tablename__ = "product"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String)
    description = Column(String)
    price = Column(Float)
    quantity = Column(Integer)
```

dbConfig.py

```
from sqlalchemy.orm import sessionmaker
from sqlalchemy import create_engine

db_url = "postgresql://postgres:12345678@localhost:5432/FASTAPI_Practice"

engine = create_engine(db_url)

Session = sessionmaker(autocommit=False, autoflush=False, bind=engine)
```

main.py

```
from fastapi import Depends, FastAPI
from models import Product
from dbConfig import Session, engine
from sqlalchemy.orm import Session as DBSession
import dbmodels

app = FastAPI()

dbmodels.Base.metadata.create_all(bind=engine)

@app.get("/")
def greet():
    return "Welcome to my website"

products = [
    Product(id=1, name="Phone", description="A smartphone", price=699.99,
quantity=50),
    Product(id=2, name="Laptop", description="A powerful laptop", price=999.99,
quantity=30),
    Product(id=3, name="Pen", description="A blue ink pen", price=1.99,
quantity=100),
    Product(id=4, name="Table", description="A wooden table", price=199.99,
quantity=20),
]

def get_db():
    db = Session()
    try:
        yield db
    finally:
        db.close()

def db_init():
    db = Session()

    count = db.query(dbmodels.Product).count()
    if count == 0:
        for product in products:
            db.add(dbmodels.Product(**product.model_dump()))
```

```
        db.commit()

db_init()

# Read
@app.get("/products")
def getAllProducts(db: DBSession = Depends(get_db)):
    db_products = db.query(dbmodels.Product).all()

    return db_products

# Specific Read
@app.get("/product/{id}")
def getProductById(id: int, db: DBSession = Depends(get_db)):
    db_product = db.query(dbmodels.Product).filter(dbmodels.Product.id ==
id).first()
    if db_product:
        return db_product

    return "Product not found"

# Create
@app.post("/product")
def addProduct(product: Product, db: DBSession = Depends(get_db)):
    db.add(dbmodels.Product(**product.model_dump()))
    db.commit()
    return product

# Update
@app.put("/product")
def updateProduct(id: int, product: Product, db: DBSession = Depends(get_db)):
    db_product = db.query(dbmodels.Product).filter(dbmodels.Product.id ==
id).first()
    if db_product:
        db_product.name = product.name
        db_product.description = product.description
        db_product.price = product.price
        db_product.quantity = product.quantity
        db.commit()
        return "Product updated Successfully"
    else:
        return "Product Not Found"

# Delete
@app.delete("/product")
def deleteProduct(id: int, db: DBSession = Depends(get_db)):
    db_product = db.query(dbmodels.Product).filter(dbmodels.Product.id ==
id).first()
    if db_product:
        db.delete(db_product)
        db.commit()
        return "Product Deleted"
```

```
    else:
        return "Product Not found"
```

---

16. Things To Fix / Watch Out For

These are small bugs currently sitting in my practice `main.py` — worth remembering:

1. **`.count` missing parentheses** in `db_init()`. `db.query(dbmodels.Product).count` is the method object, not the number. Should be `.count()`.
2. **Stray `return "No product Found"`** left at module level, just after the commented-out `updateProduct` block. A `return` outside a function is a `SyntaxError` in Python — it must be inside the comment block or deleted.
3. **`updateProduct` in my old notes was missing `db.commit()`** — the actual `main.py` has it. Without the commit, nothing is saved because `autocommit=False`.
4. **Database password is hardcoded** in `dbConfig.py`. Fine for practice, but real projects put it in an environment variable / `.env` file.
5. **`id` is accepted from the client** in the Pydantic `Product` model on POST. Normally the database should generate the ID itself.

---

17. Appendix: Markdown Cheat Sheet

Since I wasn't sure how I made the headings — this is how Markdown works:

What I want	What I type
Big heading	`# Heading 1`
Sub heading	`## Heading 2`
Smaller sub heading	`### Heading 3`
Bold	`**bold**`
Italic	`*italic*`
`inline code`	`` `code` ``
Bullet list	`- item`
Numbered list	`1. item`
Quote / note box	`> note`
Horizontal line	`---`
Link	`[text](https://url)`

Code block — three backticks, then the language name, then the code, then three backticks again:

````
```python
print("hello")
```
````

The language name (`python`, `bash`, `text`, `sql`) is what gives the code its colours.

Table — pipes for columns, dashes for the header line:

```
| Column A | Column B |
| --- | --- |
| value 1 | value 2 |
```

> **Tip:** In VS Code press `Ctrl + Shift + V` to preview a `.md` file and see it rendered.